Architekturspezifikation und Entwickler-Handoff: Bundeswehr-Dienstplan SaaS-Plattform v1.0

1. Zweck, Geltungsbereich und Status

Diese technische Spezifikation konsolidiert die fachlichen, rechtlichen und systemarchitektonischen Anforderungen für den Aufbau einer mandantenfähigen Dienstplan-SaaS-Plattform für die Bundeswehr. Das Dokument dient als verbindliche Grundlage für das Engineering-Team und überträgt die architektonische Tiefe etablierter SaaS-Refenzmodelle auf die komplexen Rahmenbedingungen des militärischen Schicht- und Dienstbetriebs.

Die Plattform löst die Kernaufgabe, Dienstpläne unter strikter Einhaltung des Soldatengesetzes (§ 30c SG) sowie der Soldatenarbeitszeitverordnung (SAZV) automatisiert zu generieren, zu validieren und bereitzustellen. Das System deckt dabei sowohl den regulären Grundbetrieb mit einer wöchentlichen Arbeitszeit von 41 Stunden und ununterbrochenen Ruhezeiten von mindestens 11 Stunden ab als auch spezialisierte Schicht- und Wachdienste nach dem Unmittelbaren Zwangsrecht der Bundeswehr (UZwGBw). Darüber hinaus erfordert die militärische Zweckbestimmung die Modellierung von Ausnahmeszenarien gemäß § 30c Abs. 4 SG für Auslandseinsätze, Übungen sowie Aufgaben der Landes- und Bündnisverteidigung, bei denen arbeitsschutzrechtliche Vorgaben dynamisch ausgesetzt oder durch Sonderlogiken ersetzt werden.

Sicherheitstechnisch orientiert sich das Gesamtsystem an den IT-Grundschutz-Standards des Bundesamtes für Sicherheit in der Informationstechnik (BSI-Standards 2001 bis 2003) und erfüllt die Auflagen für die Verarbeitung von Informationen des Geheimhaltungsgrades Verschlusssache – Nur für den Dienstgebrauch (VS-NfD). Im Rahmen der architektonischen Härtung wird der OAuth 2.0 Resource Owner Password Credentials Grant ausnahmslos gestrichen und durch einen OpenID Connect (OIDC) Authorization Code Flow mit Proof Key for Code Exchange (PKCE) ersetzt, um den Vorgaben von IETF RFC 9700 zu entsprechen.

Parameter / Dimension	Vorgabe / Ausprägung	Compliance- / Rechtsgrundlage
Reguläre Arbeitszeit	41 Stunden / Woche im Durchschnitt	§ 30c SG
Tägliche Ruhezeit	Mindestens 11 Stunden ununterbrochen	§ 5 SAZV, § 5 ArbZG
Max. Tagesarbeitszeit	13 Stunden im Grundbetrieb	§ 7 SAZV
Bereitschaftsdienst-Deckel	Bis zu 48 Stunden / Woche im Mittel	§ 12 SAZV
Sanitätsdienst-Höchstgrenze	Bis zu 54 Stunden / Woche (mit Opt-Out)	§ 13 SAZV
Sicherheitsstandard	BSI IT-Grundschutz / VS-NfD	BSI-Standards 2001-2003,

Parameter / Dimension	Vorgabe / Ausprägung	Compliance- / Rechtsgrundlage
	konform	BMVg-Vorgaben
Identity & Access	OIDC + OAuth 2.0 Authorization Code + PKCE	IETF RFC 9700 BCP

2. Architekturentscheidungen (ADRs)

ADR-ID	Architekturentscheidung	Begründung und systemische Konsequenz
ADR-01	Entkopplung von API-Gateway und dediziertem Optimierungs-Service	Das FastAPI-Backend übernimmt REST/OpenAPI-Endpunkte, Authentifizierung und Mandantenkontext. Die mathematische Schichtoptimierung wird an einen entkoppelten Engine-Service auf Timefold-Solver-Basis ausgelagert.
ADR-02	Verwendung von Timefold Solver mit ConstraintStreams API	Anstelle stochastischer LLM-Generierung kommt ein deterministischer Constraint Solver zum Einsatz. Er garantiert die harte Trennung zwischen rechtlichen Abbruchkriterien und operativen Zielvorgaben.
ADR-03	PostgreSQL Row-Level Security (RLS) mit FORCE ROW LEVEL SECURITY	Verhindert Broken Object Level Authorization (BOLA) auf Datenbankebene. Der Anwendungsbenutzer besitzt kein BYPASSRLS; Mandanten-IDs werden transaktionslokal gesetzt.
ADR-04	LangGraph-Orchestrator für Befehls- und Freigabeworkflows	Dienstpläne stellen Befehle nach § 10 SG dar. LangGraph verwaltet zustandsbehaftete Interrupts für Kompaniechef-Freigaben mit persistenten PostgreSQL-Checkpoints.
ADR-05	Gehärtete Sandbox-Infrastruktur für Regelskripte	Benutzerdefinierte Auswertungsskripte laufen ausschließlich in rootless gVisor- oder MicroVM-Containern ohne

ADR-ID	Architekturentscheidung	Begründung und systemische Konsequenz
		Docker-Socket-Mounts oder internen Netzwerkzugriff.
ADR-06	Dezentrale Edge-Synchronisation mit CRDTs	Garantiert den schreibgeschützten Zugriff sowie eingeschränkte Interaktionen (z. B. Tauschanfragen) auf mobilen Endgeräten in Feldlagern mit instabiler Netzanbindung.
ADR-07	Transactional Outbox Pattern für Zeiterfassungs-Exporte	Verhindert Dateninkonsistenzen bei Abrechnungsexporten durch atomare Speicherung von DATEV-Buchungsstapeln in der SQL-Datenbank vor der Übertragung.
ADR-08	Datenschutzkonforme Löschkaskade statt Soft-Delete	Erfüllt Art. 17 DSGVO und soldatenrechtliche Aufbewahrungsfristen durch Bereinigung aktiver DBs, Redis-Caches und S3-Storage sowie Führung einer Deletion Registry.

Die gewählten Architekturentscheidungen greifen direkt ineinander. Die Abgrenzung der Optimierungs-Engine stellt sicher, dass ressourcenintensive Berechnungen der Schichtzuweisungen das API-Gateway nicht blockieren. Gleichzeitig verzahnt sich die LangGraph-Workflow-Steuerung eng mit dem Datenmodell: Sobald der Constraint Solver einen gültigen Dienstentwurf berechnet hat, versetzt der Orchestrator den Ablauf in einen Interrupt-Zustand. Der Dienstplan verbleibt in diesem Schwebezustand, bis der zuständige Vorgesetzte die Einteilung nach § 10 SG digital unterzeichnet. Die erzwungene Datenbank-Isolierung via Row-Level Security verhindert dabei zuverlässig, dass Führungskräfte während des Genehmigungsprozesses Einsicht in Dienstpläne fremder Einheiten erhalten.

3. Zielarchitektur und Bausteine

Die Gesamtsystemarchitektur unterteilt sich in klar abgegrenzte Schichten, die über definierte Netzwerkgrenzen und Sicherheitsprotokolle miteinander kommunizieren.

Client-Schicht und Zugriffspfade

Das System bietet zwei primäre Zugriffspfade. Für Planer, Truppenärzte und Vorgesetzte steht eine browserbasierte Webanwendung bereit, die komplexe Matrix-Darstellungen und Simulationswerkzeuge bereitstellt. Für Soldatinnen und Soldaten existiert eine progressive Web-Applikation (PWA) für mobile Endgeräte. Diese Schicht kommuniziert ausschließlich über verschlüsselte HTTPS- und WebSocket-Verbindungen mit dem API-Gateway und nutzt

OIDC-Token für die Autorisierung.

API-Gateway und Applikationslogik (FastAPI)

Das API-Gateway auf FastAPI-Basis verarbeitet eingehende REST-Anfragen, validiert Eingabedaten über Pydantic-v2-Schemas und setzt den Autorisierungskontext durch Prüfung der im JWT enthaltenen Rollen und Mandanten-Zuordnungen durch. Anstelle direkter Datenbankzugriffe aus der Geschäftslogik wird bei jeder eingehenden Anforderung der transaktionslokale Mandantenkontext in der Datenbank-Session verankert.

Optimierungs- und Solver-Schicht (Timefold Solver Engine)

Die mathematische Generierung von Dienstplänen ist als eigenständiger Mikroservice in Java/Spring Boot oder Python ausgeführt, welcher die Timefold-Solver-Bibliothek einbettet. Dieser Service nimmt strukturierte JSON-Problembeschreibungen entgegen, führt die Schichtzuweisung anhand der definierten Hard-, Medium- und Soft-Constraints durch und gibt das optimierte Arbeitsraster inklusive Score-Analyse an den Orchestrator zurück.

Orchestrierung und Worker-Pool (LangGraph & Taskiq/Celery)

Länger laufende Aufgaben wie die solverbasierte Generierung, die Generierung von Exportdateien sowie die Ausführung von Benachrichtigungen werden asynchron über Worker-Pools abgewickelt. Zustandssensitive Mehrschritt-Workflows werden von LangGraph gesteuert. Die Zustände der Ausführungs-Graphen werden transaktional in PostgreSQL persistiert, um Abstürze von Worker-Knoten ohne Datenverlust zu überstehen.

Datenebene (PostgreSQL, Redis, Object Storage)

Die relationale Datenbasis PostgreSQL speichert Stamm- und Bewegungsdaten, Arbeitszeitkonten, Dienstpläne sowie Audit-Logs. Die Ausführung von RLS verhindert mandantenübergreifende Datenzugriffe. Redis wird strikt auf flüchtiges Caching, distributed Locking sowie Rate-Limiting beschränkt. Das S3-kompatible Object Storage verwaltet binäre Artefakte, exportierte PDF-Dokumente und verschlüsselte Audit-Pakete.

Sandbox-Service

Für die Ausführung externer Regelskripte oder erweiterter Analysen kommt ein dedizierter Sandbox-Dienst zum Einsatz. Dieser läuft auf isolierter Infrastruktur und nutzt Container-Isolierungstechnologien wie gVisor ohne Verbindung zum internen Anwendungsnetzwerk.

4. Laufzeitsicht, Workflow-Zustände und Engine-Orchestrierung

Ein Dienstplan in der Bundeswehr ist kein unverbindlicher Schichtvorschlag, sondern die Ausprägung militärischer Befehlsgebung gemäß § 10 SG. Dementsprechend unterliegt der

Erstellungs- und Freigabeprozess strengen verfahrensrechtlichen Zustandsübergängen.

Detaillierter Zustandsablauf

1. Empfang des Erstellungsauftrags (DRAFT_INIT): Der Dienstplaner definiert den Planungszeitraum, die betroffene Einheit sowie operative Ausnahmezustände (z. B. aktiver Übungsbetrieb nach § 20 SAZV).
2. Datenaggregation (DATA_FETCHING): Das System liest Personalverfügbarkeiten, bestehende Urlaubseinträge, Krankmeldungen, Lehrgangsbuchungen und Vorkonfigurationen aus der PostgreSQL-Datenbank aus.
3. Vorvalidierung (CONSTRAINT_VALIDATION): Die Vorbedingungen werden geprüft. Fehlende Mindestqualifikationen für Schlüsselpositionen (z. B. Wachhabender) erzeugen sofortige Hinweismeldungen.
4. Solver-Optimierung (SOLVER_OPTIMIZING): Die Daten werden in das Timefold-Problemmodell überführt. Die Optimierungs-Engine berechnet unter kontinuierlicher Auswertung der ConstraintStreams die optimale Schichtzuordnung.
5. Feasibility-Prüfung (HARDFEASIBILITY_CHECK): Das System prüft, ob der erzeugte Plan frei von Hard-Constraint-Verletzungen ist. Weist der Plan negative Hard-Scores auf, schlägt der automatische Prozess fehl und führt den Planer in eine manuelle Nachbearbeitung.
6. Simulations- & Belastungsprüfung (SIMULATION_REVIEW): Der Planer analysiert die Auswirkungen des Entwurfs auf die Arbeitszeitkonten, den Freizeitausgleich (§ 22, § 23 SAZV) und die faire Verteilung von Nachtdiensten.
7. Vorgesetzten-Freigabe (COMMAND_APPROVAL_INTERRUPT): Der LangGraph-Orchestrator unterbricht den Prozess und wartet auf die explizite Freigabe durch den Einheitsführer (Human-in-the-Loop).
8. Veröffentlichung & Befehlserlass (PUBLISHED): Nach digitaler Autorisierung durch den Einheitsführer wird der Dienstplan rechtsverbindlich, in die Datenbank geschrieben und für die Soldaten einsehbar geschaltet.
9. Echtzeit-Re-Optimierung (REALTIME_REPLANNING): Bei Spontanausfällen im laufenden Betrieb (z. B. Krankmeldung) führt das System eine Nachoptimierung durch. Bereits begonnene oder kurz bevorstehende Schichten werden dabei unveränderlich fixiert (Pinning).

Regeln für LangGraph-Orchestrierung und Interrupts

Für die Steuerung der Workflows gilt, dass lang laufende Graphen mit persistenten PostgreSQL-Checkpointern und einer stabilen thread_id betrieben werden müssen. Da Wiederaufnahmen nach einem Interrupt dazu führen können, dass einzelne Knoten erneut ausgeführt werden, müssen sämtliche Seiteneffekte vor dem Interrupt vollkommen idempotent gestaltet sein. Nicht-idempotente Aktionen, wie das Versenden von Benachrichtigungen oder das Auslösen externer Buchungszeilen, sind ausnahmslos in Knoten zu verlegen, die nach der Freigabe liegen. Sämtliche Interrupt-Payloads werden streng nach Pydantic-Schemas typisiert und vor der Verarbeitung im Graph-State validiert.

5. Optimierungs-Engine & Constraint-Modellierung

(Timefold/ConstraintStreams)

Die automatisierte Erstellung von Schichtplänen erfordert die Bewältigung NP-harter kombinatorischer Probleme. LLMs sind aufgrund ihrer stochastischen Natur ungeeignet für die Einhaltung exakter rechtlicher Höchstgrenzen. Das System nutzt daher Timefold Solver, welches auf objektorientierter Modellierung und inkrementeller Score-Berechnung über Constraint Streams basiert.

Domänen-Modellierung

Das Domänenmodell gliedert sich in drei Kernbestandteile:

- Problem Facts: Unveränderliche Fakten wie Soldaten, deren Qualifikationen, Abwesenheiten und gesetzliche Bestimmungen.
- Planning Entities: Objekte, die während des Lösungsprozesses verändert werden. Die zentral definierte Planungseinheit ist die Schicht (Shift), deren Zuweisungsvariable (assignedEmployee) vom Solver modifiziert wird.
- Planning Solution: Repräsentiert den Gesamtzustand des Dienstplans (ShiftSchedule) inklusive aller Schichten, Soldaten und des aktuellen Gesamt-Scores.

Constraint Matrix (SAZV / Soldatengesetz)

Die Zielvorgaben werden über mehrstufige Score-Systeme (HardSoftScore oder HardMediumSoftScore) abgebildet. Ein Plan ist nur dann zulässig (feasible), wenn der Hard-Score exakt 0 beträgt.

Constraint-Bezeichnung	Ebene	Legal- / Betriebsgrundlage	Implementierungslogik (ConstraintStreams API)
ruhezeitEinhaltung	Hard	§ 5 SAZV, § 5 ArbZG	Bestraft Schichtpaare desselben Soldaten, wenn der zeitliche Abstand zwischen dem Ende der ersten Schicht und dem Beginn der Folge-Schicht weniger als 11 Stunden beträgt.
maxTagesarbeitszeit	Hard	§ 7 SAZV	Gruppiert Schichten pro Soldat und Kalendertag und bestraft Überschreitungen der täglichen Höchstarbeitszeit von 13 Stunden im Grundbetrieb.
erforderlicheQualifikation	Hard	UZwGBw / Dienstvorschrift	Vergleicht die geforderte Schicht-Qualifikation

Constraint-Bezeichnung	Ebene	Legal- / Betriebsgrundlage	Implementierungslogik (ConstraintStreams API)
			mit dem Skill-Set des zugewiesenen Soldaten und penalisiert fehlende Übereinstimmungen.
keineDoppelbelegung	Hard	Logische Grundregel	Penalisiert Überschneidungen zeitlicher Schichtfenster bei demselben Soldaten.
ausnahmeParagraph30c	Hard	§ 30c Abs. 4 SG	Inaktiviert Standard-Arbeitszeitgrenzen bei aktiver Einsatz- oder Übungs-Flagge und aktiviert angepasste Sonderregeln.
bereitschaftsDeckelung	Medium	§ 12 SAZV	Überwacht die Einhaltung der maximalen Wochenarbeitszeit von 48 Stunden bei hohen Bereitschaftsanteilen im Mehrwochendurchschnitt.
faireBelastungsverteilung	Soft	Moralerhaltung / Fairness	Minimiert die Varianz der Gesamtarbeitsstunden und Nachtdienste im Soldatenteam über den Planungszeitraum.
soldatenWunsch	Soft	Moralerhaltung / Dienstzufriedenheit	Belohnt die Übereinstimmung von Schichtzuweisungen mit eingereichten Freistellungs- oder Präferenzmeldungen.

Das folgende Implementierungsbeispiel zeigt die Definition rechtlicher Regeln mittels der ConstraintStreams API in Java:

```
public class SAZVConstraintProvider implements ConstraintProvider {
    @Override
    public Constraint[]
de[span_33](start_span)[span_33](end_span)fineConstraints(ConstraintFa
```

```
ctory constraintFactory) {
        return new Constraint[] {
            ruhezeitEinhaltung(constraintFactory),
            erforderlicheQualifikation(constraintFactory),
            maxTagesarbeitszeit(constraintFactory)
        };
    }

    private Constraint ruhezeitEinhaltung(ConstraintFactory
constraintFactory) {
        return constraintFactory
      [span_63](start_span)[span_63](end_span)
.forEachUniquePair(Shift.class,
                Joiners.equal(Shift::getAssignedEmployee),
                Joiners.lessThan(Shift::getEndTime,
Shift::getStartTime))
            .filter((shift1, shift2) ->
                Duration.between(shift1.getEndTime(),
shift2.getStartTime()).toHours() < 11)
            .penalize(HardSoftScore.ONE_HARD)
            .asConstraint("Ruhezeit von 11h unterschritten");
    }

    private Constraint erforderlicheQualifikation(ConstraintFactory
constraintFactory) {
        return constraintFactory
            .forEach(Shift.class)
            .filter(shift -> shift.getAssignedEmployee() != null &&

!shift.getAssignedEmployee().getSkills().contains(shift.getRequiredSki
ll()))
            .penalize(HardSoftScore.ONE_HARD)
            .asConstraint("Fehlende Qualifikation für Schicht");
    }

    private Constraint maxTagesarbeitszeit(ConstraintFactory
constraintFactory) {
        return constraintFactory
            .forEach(Shift.class)
            .groupBy(Shift::getAssignedEmployee,
                     shift -> shift.getStartTime().toLocalDate(),
                     ConstraintCollectors.sum(shift -> (int)
Duration.between(shift.getStartTime(), shift.getEndTime()).toHours()))
            .filter((employee, date, totalHours) -> totalHours > 13)
            .penalize(HardSoftScore.ONE_HARD)
            .asConstraint("Tägliche Höchstarbeitszeit von 13h
überschritten");
    }
```

```
}
```

6. Multi-Tenancy, Datenmodell und Datenbank-Sicherheit

Die Architektur setzt auf eine mandantenfähige Multi-Tenant-Struktur. Jede militärische Dienststelle bzw. Kompanie fungiert als isolierter Tenant, um Vertraulichkeit und Truppengeheimhaltung zu gewährleisten.

PostgreSQL Row-Level Security (RLS)

Zur Verhinderung von Cross-Tenant-Datenlecks wird RLS direkt im PostgreSQL-Server erzwungen. Der Anwendungsbenutzer greift ohne BYPASSRLS-Privilegien auf die Datenbank zu. Das Schema erzwingt FORCE ROW LEVEL SECURITY für alle mandantenbezogenen Tabellen.

```
-- Erstellung und Aktivierung der RLS-Sicherung auf der
Dienstplan-Tabelle
ALTER TABLE dienstplan ENABLE ROW LEVEL SECURITY;
ALTER TABLE dienstplan FORCE ROW LEVEL SECURITY;

ALTER TAB[span_74](start_span)[span_74](end_span)LE soldat ENABLE ROW
LEVEL SECURITY;
ALTER TABLE soldat FORCE ROW LEVEL SECURITY;

-- Mandanten-Isolierungspolicy basierend auf transaktionslokaler
Konfiguration
CREATE POLICY tenant_isolation_dienstplan ON dienstplan
    FOR ALL
    TO application_user
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid)
    WITH CHECK (tenant_id =
current_setting('app.current_tenant_id')::uuid);

CREATE POLICY tenant_isolation_soldat ON soldat
    FOR ALL
    TO application_user
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid)
    WITH CHECK (tenant_id =
current_setting('app.current_tenant_id')::uuid);
```

Transaktionslokale Kontext-Injektion

Um zu verhindern, dass bei Wiederverwendung von Datenbankverbindungen aus dem Connection-Pool der Kontext vorheriger Aufrufe erhalten bleibt, injiziert das Backend die Mandanten-ID transaktionslokal mittels SET LOCAL:

```
# Async Database Dependency in FastAPI
async def get_db_session(current_user: AuthenticatedUser =
Depends(get_current_user)):
    async with AsyncSessionLocal() as session:
        async with session.begin():
            # Transaktionslokale Setzung des Tenant-Kontexts für
Postgres RLS
            await session.execute(
                text("SET LOCAL app.current_tenant_id = :tenant_id"),
                {"tenant_id": str(current_user.tenant_id)}
            )
            yield session
```

Kernelemente des relationalen Datenmodells

Das Datenmodell verzahnt die militärische Organisationsstruktur mit den arbeitszeitrechtlichen Vorgaben:

- tenants: Enthält die Stammdaten der Einheiten (z. B. Kompanie, Bataillon) inklusive Sicherheits-Klassifizierung.
- soldat: Speichert Personenstammdaten, Dienstgrad, Personalnummer, zugeordnete Qualifikationen (ATNs) sowie Referenzen auf Salden der Arbeitszeitkonten.
- dienstposten_schicht: Definiert das Soll-Personal für einen Dienst inklusive geforderter ATNs, Dienstzeiten und Anrechnungsfaktoren nach SAZV.
- dienstplan: Das Aggregat eines Schichtplans für einen Zeitraum. Es hält den Freigabestatus (DRAFT, APPROVED), den Referenzschlüssel auf den Vorgesetzten (approver_id) sowie die Befehls-Audit-Spur.
- arbeitszeitkonto: Führt Historien über geleistete Dienststunden, Mehrarbeit, Bereitschaftsdienstanteile sowie Freizeitausgleiche gemäß § 30c SG.

7. Sicherheitsarchitektur, BSI IT-Grundschutz & Verschlusssachen (VS-NfD)

Die Verarbeitung von Personaldaten und Truppenstärken erfordert die Einhaltung eines Informationssicherheits-Managementsystems (ISMS) nach BSI IT-Grundschutz (BSI-Standards 2001 bis 2003) sowie des IT-Grundschutz-Kompendiums.

IT-Grundschutz Baustein-Mapping

Grundschutz-Baustein	Anwendungsbereich	Technische & Organisatorische Maßnahme (TOM)
ORP.4: Identitäts- und Rechtemanagement	RBAC, Auth, Mandantentrennung	Multi-Faktor-Authentifizierung (MFA), OAuth 2.0 PKCE, strikte Session-Trennung, Postgres RLS.
CON.2: Datenschutz	Personenbezogene Daten	Zweckgebundene Verarbeitung,

Grundschutz-Baustein	Anwendungsbereich	Technische & Organisatorische Maßnahme (TOM)
		Argon2id für Passwort-Hashes, AES-256-GCM für Data-at-Rest.
OPS.1.1.5: Protokollierung	Dienstaufsicht & Audit	Unveränderbare, zentrale Audit-Logs aller Dienstplanänderungen und Vorgesetzten-Freigaben.
APP.3.1: Webanwendungen	API Gateway & UIs	Strict Content Security Policy (CSP), OWASP-Mitigation, Verhinderung von BOLA via RLS.
SYS.1.6: Containerisierung	Sandboxing & Microservices	Rootless Runner, gVisor Runtime, Egress-Deny, schreibgeschützte Filesysteme.

Sandbox-Härtung für unvertraute Skripte

Sollten im Rahmen von Sonderauswertungen benutzerdefinierte Regelskripte zur Ausführung kommen, greifen strikte Sandbox-Auflagen:

- Das Anwendungssystem erhält keinen Zugriff auf den Docker-Socket des Hosts, um Container-Escape-Angriffe auszuschließen.
- Die Ausführung erfolgt in isolierten microVM- oder gVisor-Runnern (runsc) mit dediziertem Kernel-Space.
- Das Netzwerk der Sandbox ist standardmäßig vollständig blockiert (Egress Deny); externer Zugriff erfolgt ausschließlich über explizite Proxy-Allowlists.
- Es gelten Ressourcen-Limits: Maximal 512 MB RAM, maximal 30 Sekunden CPU-Laufzeit, Begrenzung der Prozessanzahl und Verbot von Root-Rechten.

8. Identity, Auth, API-Spezifikation & Role-Based Access Control (RBAC)

Auth-Architektur (RFC 9700 BCP Konformität)

Der Einsatz des *OAuth2 Password Flow* ist untersagt. Die Authentifizierung wird über einen zentralen OpenID-Connect-Provider mittels *Authorization Code Flow mit PKCE* abgewickelt. Access Tokens sind auf 15 Minuten beschränkt. Refresh Tokens nutzen token rotation mit automatischer Replay Detection, bei der die Wiederverwendung eines bereits genutzten Refresh Tokens zum sofortigen Widerruf der gesamten Session-Familie führt.

API-Spezifikation (FastAPI Endpunkte)

Methode	Endpunkt	Rollen / RBAC	Zweck & Sicherheitsauflagen
POST	/api/v1/jobs/roster	Planer, Admin	Initiierung eines asynchronen

Methode	Endpunkt	Rollen / RBAC	Zweck & Sicherheitsauflagen
			Optimierungslaufs. Erfordert Idempotency-Key und Tenant-Scope.
GET	/api/v1/jobs/roster/{id}	Planer, Audit	Abruf von Status und Zwischenergebnissen. Zugriff wird über RLS abgesichert.
POST	/api/v1/jobs/roster/{id}/approve	Kommandeur	Digitale Freigabe des Dienstplans (§ 10 SG). Löst LangGraph Resume aus und schreibt Audit-Log.
POST	/api/v1/jobs/roster/{id}/cancel	Planer, Admin	Abbruch des Solver-Laufs. Stoppt Hintergrund-Prozesse.
GET	/api/v1/roster/my-shifts	Soldat	Abruf eigener Schichten und Ruhezeiten. Optimiert für Caching auf mobilen Endgeräten.
POST	/api/v1/roster/trades	Soldat	Initiierung einer Diensttauschanfrage an einen Kameraden.
POST	/api/v1/exports/datev	Admin, Audit	Generierung von Datev-ASCII-Buchungsstapeln über Transactional Outbox Pattern.
DELETE	/api/v1/users/me	Soldat	Start des DSGVO-Art.-17-Löschprozesses unter Einhaltung gesetzlicher Fristen.

9. Integrationen, Zeiterfassung & DATEV-Schnittstelle

Das Dienstplansystem stellt Schnittstellen zu Zeiterfassungssystemen der Bundeswehr sowie zu Abrechnungssystemen wie DATEV bereit.

DATEV ASCII-Format Spezifikation

Beim Export von Zeiterfassungsdaten müssen die Unterschiede zwischen **DATEV LODAS** und **DATEV Lohn und Gehalt** beachtet werden. Ein für ein System generierter Export kann im jeweils anderen System nicht verarbeitet werden.

Format-Eigenschaft	DATEV LODAS Spezifikation	DATEV Lohn und Gehalt Spezifikation
Kopfzeilen-Aufbau	Kopfteil mit Zielsystem, Berater- & Mandantennummer, Satzbeschreibung	Spezifischer Header inkl. HR/SR-Kennung
Datumsformat	DD.MM.YYYY (z. B. 01.04.2026)	MM/YYYY bzw. Tagesdatum je nach Feldtyp
Dezimaltrenner	Dezimalkomma (,)	Dezimalpunkt (.)
Spaltenreihenfolge	Personalnummer;Datum;Stunden;Lohnart;Bearbeitungsschlüssel	Personalnum[span_175](start_span)[span_175](end_span)mer;Datum;Art;Stunden;Tagesfaktor
Zeichensatz	Windows-1252 / ISO-8859-1 (ASCII, Umlaute konvertiert)	Windows-1252 / ASCII

Transaktionssicherer Transport (Transactional Outbox)

Exportdateien werden über das Transactional Outbox Pattern verarbeitet:

1. Bei Auslösung eines Exports schreibt die API die Bewegungsdaten atomar zusammen mit dem Geschäftsstatus in die Datenbanktabelle datev_outbox.
2. Ein Hintergrund-Worker liest neue Outbox-Einträge aus, gleicht die internen Abwesenheitsarten mit den von der Kanzlei vorgegebenen Lohnarten ab und formatiert die ASCII-Datei.
3. Die Übertragung erfolgt wahlweise per Lohnimportdatenservice über die DATEV-Cloud oder als DATEV-ASCII-Datei.

10. Offline-Fähigkeit, Synchronisation & Resilienz

Der Einsatz in Feldlagern oder auf See erfordert temporäre Offline-Lauffähigkeit.

Edge-Synchronisation und Konfliktbehandlung

Mobile Endgeräte und lokale Server vor Ort halten eine lokale Read-Only-Kopie der Dienstpläne auf Basis von SQLite oder IndexedDB. Schreibaktionen im Offline-Modus sind auf nicht-befehlsverändernde Aktionen wie persönliche Verfügbarkeitsmeldungen oder Diensttausch-Gesuche beschränkt. Die Re-Synchronisation nutzt Optimistic Concurrency Control und Conflict-Free Replicated Data Types (CRDTs). Bei Konflikten zwischen manuellen Tauschanfragen und Befehlsänderungen durch Vorgesetzte besitzt die Entscheidung des Vorgesetzten gemäß § 10 SG stets Vorrang.

Rate-Limiting via Redis Token Bucket

Zum Schutz des Gateway-Systems vor Überlastung wird ein Redis-basiertes Rate-Limiting eingesetzt:

```
-- Lua-Skript für atomaren Token-Bucket-Algorithmus in Redis
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window =
```

```
tonumber(ARGV[2])[span_75](start_span)[span_75](end_span)

local current = tonumber(redis.call('get', key) or "0")

if current + 1 > limit then
    return 0
else
    redis.call("INCR[span_95](start_span)[span_95](end_span)BY", key,
1)
    if current == 0 then
        redis.call[span_76](start_span)[span_76](end_span)("EXPIRE",
key, window)
    end
    return 1
end
```

11. Datenschutz (DSGVO) und Dienstaufsicht

Die Verarbeitung von Gesundheitsdaten (Krankmeldungen) und Truppenstärken erfordert technisch verankerten Datenschutz (Privacy by Design).

Betroffenenrechte und Lösch-Pipeline

Zur Erfüllung von Art. 15 DSGVO stellt das System unter /api/v1/users/me/export einen maschinenlesbaren JSON/PDF-Export aller zur Person gespeicherten Zeiten und Dienstpläne bereit. Die Ausübung des Rechts auf Löschung (Art. 17 DSGVO) wird nicht durch ein bloßes Soft-Delete-Flag gelöst. Stattdessen werden Daten nach Ablauf soldatenrechtlicher Aufbewahrungsfristen durch eine Lösch-Pipeline (Deletion Registry) aus aktiven Datenbanken, Caches und S3-Artefakten entfernt.
In unveränderbaren Backups verbleibende Datenbestände werden mittels Crypto-Shredding (Löschen der spezifischen Verschlüsselungsschlüssel) unbrauchbar gemacht oder bei einem System-Restore durch Abgleich mit der Deletion Registry automatisiert herausgefiltert.

Audit-Trail für die Dienstaufsicht

Änderungen an Dienstplänen, manuelles Übersteuern von Regeln sowie Befehlsfreigaben werden in einem auditierbaren Protokoll erfasst. Diese Protokolle dienen der militärischen Dienstaufsicht zur Nachvollziehbarkeit von Anordnungen nach § 10 SG. Sie enthalten keine unnötigen personenbezogenen Daten und werden verschlüsselt aufbewahrt.

12. Nichtfunktionale Anforderungen (NFRs)

Bereich	Zielwert / Akzeptanzkriterium	Verifikations- und Messmethode
API-Latenz (Synchron)	P_{95} < 300\text{ ms} für synchrone API-Routen unter	Automatisierte k6 Load-Testing-Suite in der

Bereich	Zielwert / Akzeptanzkriterium	Verifikations- und Messmethode
	Volllast.	CI/CD-Pipeline.
Solver-Konvergenzzeit	P_{95} < 30\text{ s} für eine Kompanie (150 Soldaten, 30 Tage, ~1000 Schichten).	Timefold Benchmark Test-Suite.
Mandanten-Isolierung	0 Datenübertritte in automatisierten BOLA- und Cross-Tenant-Tests.	Security Integration Pipeline mit RLS-Fuzzing.
Verfügbarkeit	99,9 % Ausfallsicherheit für Lese-Endpunkte und Mobile-Sync.	Multi-AZ Kubernetes Deployment Monitoring.
Edge-Sync-Recovery	Atomare Re-Synchronisation nach Netzausfall in < 5\text{ s}.	Chaos-Engineering-Tests (Netzwerk-Interruption).
Constraint-Präzision	0 verbleibende Hard-Constraint-Verletzungen in veröffentlichten Plänen.	Automatischer ConstraintVerifier-Stopp im Workflow.

13. Test-, Validierungs- und Freigabestrategie

Das Testkonzept umfasst mehrere aufeinander aufbauende Ebenen:

Unit- und Schema-Tests

Testen der Pydantic-v2-Modelle, Datums-Helfer sowie der LangGraph-Zustandsübergänge auf Korrektheit.

Timefold ConstraintVerifier Tests

Sämtliche rechtlichen und betrieblichen Regeln werden über isolierte ConstraintVerifier-Tests validiert. Dabei wird geprüft, ob einzelne Schicht-Konstellationen exakt die erwartete Pönalisierung im Score auslösen:

```
@Test
void ruhezeitUnterschrittenSollteHardScorePenalisieren() {

constraintVerifier.verifyThat(SAZVConstraintProvider::ruhezeitEinhaltung)
        .given(shiftEnde22Uhr, shiftStart06UhrFolgetag)
        .penalizesBy(1);
}
```

Sicherheits- und RLS-Integrationstests

Es werden automatisierte SQL-Integrationstests ausgeführt, die gezielt versuchen, unter der Identität von Mandant A Daten von Mandant B abzufragen. Das PostgreSQL-Backend muss diese Versuche durch aktives RLS verlässlicherweise verweigern.

Resilienz- und E2E-Tests

Simulation von Abbruchszenarien während eines laufenden Optimierungslaufs. Das System muss die letzte valide State-Checkpoints aus PostgreSQL lesen und den Prozess ohne Beschädigung des Datenbestandes fortsetzen.

14. Umsetzungsplan und Phasenmodell

Phase	Meilenstein / Umfang	Akzeptanzkriterium
Phase 0: Grundentscheidungen	Erstellung Threat Model, BSI-Schutzbedarfsfeststellung, ADR-Freigabe.	Akzeptierte ADRs und abgestimmtes Dateninventar.
Phase 1: Plattformfundament	Repository-Setup, FastAPI-Skelett, PostgreSQL mit Alembic, Redis, Logging.	Deploybar in Dev-Umgebung; Migrations-Rollbacks getestet.
Phase 2: Identity & Multi-Tenancy	OIDC-Integration (PKCE), Tenant-Schema, PostgreSQL RLS Policies, RBAC.	Cross-Tenant-Tests erfolgreich; DB-Pool-Reuse RLS-sicher.
Phase 3: Optimization Core Engine	Timefold Solver Integration, ConstraintStreams für SAZV/ArbZG.	ConstraintVerifier-Testsuite grün; Solver generiert valide Pläne.
Phase 4: Workflow & HITL	LangGraph Orchestrator, Checkpointer, HITL-Interrupt für Kompaniechef-Freigabe.	Crash-Recovery und Freigabeworkflow grün.
Phase 5: Integrations-Engine	Transactional Outbox Worker, DATEV ASCII Export Adapter, Edge Sync Caching.	DATEV-Export-Tests im Test-System erfolgreich.
Phase 6: Sandboxing & Hardening	gVisor Sandbox-Runner, Rate-Limiting, Backup-/Lösch-Drills.	Penetrationstest der Sandbox und Deletion-Registry-Exekution grün.
Phase 7: Pilotbetrieb	Feldtest in ausgewählten Einheiten, Performance-Monitoring, Runbooks.	Erfolgreicher Abschluss des Pilotbetriebs vor Rollout.

15. Soll-Repository-Struktur

dienstplan-saas/ ├── apps/ │ ├── api/ # FastAPI Gateway (Routen, Auth, RBAC, Middleware) │ ├── solver-service/ # Timefold Solver Engine (Java/Spring Boot oder Python Engine) │ ├── worker/ # Async Task Workers (Outbox, DATEV Export, Benachrichtigungen) │ └── edge-relay/ # Sync-Dienst für dezentrale Feldlager / Mobile Edge Clients ├── packages/ │ ├── domain/ # Domänen-Entitäten (Soldat, Schicht, Dienstplan, SAZV-Regeln) │ ├── constraints/ # Timefold ConstraintStreams Definitions & SAZV/SG Rules │ ├── orchestration/ # LangGraph Workflows, State Machine, Checkpointer Setup │ ├── security/ # Auth, OIDC Provider Integration, Tenant Context Manager, RLS Helpers │ ├── sandbox/ # Isolation Client Wrapper (gVisor Runner Client) │ └── storage/ # PostgreSQL Engine, Alembic Migrations, S3 Access Layer ├──

migrations/ │ ├── versions/ # Alembic Schema- & RLS-Policy-Skripte │ └── env.py ├── infra/ │ ├── terraform/ # Infrastructure as Code (Rechenzentrum / Cloud Environments) │ ├── k8s/ # Kubernetes Manifeste & Service Mesh Allowlists │ └── sandbox-host/ # Gehärtete Host-Konfigurationen für gVisor Runner ├── tests/ │ ├── unit/ # Pydantic Schemas, Helper functions │ ├── solver/ # Timefold ConstraintVerifier Tests │ ├── integration/ # RLS Tenant Tests, Outbox Pattern Tests │ ├── security/ # BOLA Scans, Penetration Suite │ └── e2e/ # End-to-End Workflow-Tests ├── pyproject.toml # Python Dependencies & Toolchain Definitions ├── pom.xml # Java Dependencies (Timefold Core Engine) └── compose.yaml # Lokale Entwicklungsumgebung (Postgres, Redis, Mock-OIDC)

16. Review der ursprünglichen Vorgaben und Abgrenzung

Im Rahmen der Konsolidierung wurden bestehende Annahmen revidiert:

- Das Verfahren *OAuth2 Password Flow* wird gestrichen und durch OIDC Authorization Code + PKCE ersetzt.
- Der Einsatz von Soft-Delete zur Erfüllung von Art. 17 DSGVO ist unzureichend; es wird eine automatische Löschkaskade inkl. Deletion Registry implementiert.
- Generative Sprachmodelle werden nicht für die Schichtzuweisung genutzt. Die Schichtberechnung verbleibt bei der mathematischen Timefold-Solver-Engine.
- Die Ausführung unvertrauter Regelskripte in Docker-Containern bietet ohne Kernel-Isolierung (gVisor/MicroVM) keine hinreichende Mandantentrennung auf Host-Ebene.

17. Fazit und Handlungsempfehlungen

Die vorliegende Spezifikation überträgt die Anforderungen der Dienstplanung im Bundeswehrumfeld in eine hochverfügbare, mandantenfähige SaaS-Architektur. Durch die strikte Trennung von API-Gateway, LangGraph-Orchestrierung und der mathematischen Timefold-Optimierungs-Engine entsteht ein System, das sowohl rechtliche Vorgaben wie die SAZV als auch militärische Führungsstrukturen (§ 10 SG) rechtskonform abbildet.
Die Implementierung sollte phasenweise gemäß dem definierten Umsetzungsplan erfolgen. Besonderes Augenmerk ist in den Phasen 1 bis 3 auf die absicherungstechnischen Fundamente – insbesondere die durchgängige Durchsetzung von PostgreSQL Row-Level Security sowie die präzise Implementierung der ConstraintVerifier-Testreihen – zu legen, um eine belastbare Basis für den anschließenden Pilotbetrieb zu schaffen.

Quellenangaben

1. Timefold Solver | Technology Radar - Thoughtworks, https://www.thoughtworks.com/radar/tools/timefold-solver 2. OptaPlanner - The fast, Open Source and easy-to-use solver, https://optaplanner.io/ 3. A Guide to Timefold Solver for Employee Scheduling - Baeldung, https://www.baeldung.com/java-timefold-solver-guide 4. Programming Constraint Solver and Optimization APIs - Timefold, https://timefold.ai/constraint-optimization 5. Employee Shift Scheduling AI in Python - Timefold, https://timefold.ai/videos/employee-shift-scheduling-ai-in-python 6. Arbeitszeitgesetz (ArbZG) -

Gesetze im Internet, https://www.gesetze-im-internet.de/arbzg/BJNR117100994.html 7. Timefold: Enterprise Scheduling Optimization APIs, https://timefold.ai/ 8. Timefold Solver is an AI constraint solver for Python to ... - GitHub, https://github.com/TimefoldAI/timefold-solver-python 9. Timefold scheduling: am I running into limitations of the solver or am, https://stackoverflow.com/questions/79213284/timefold-scheduling-am-i-running-into-limitations-of-the-solver-or-am-i-doing-s 10. Timefold Solver Python live in Alpha, https://timefold.ai/blog/introducing-timefold-python-solver 11. Overview | Constraints and score | Timefold Solver 2.8.0, https://docs.timefold.ai/timefold-solver/latest/constraints-and-score/overview 12. Open Source Solver for Scheduling and Routing - Timefold, https://timefold.ai/solver 13. Zeiterfassung mit DATEV-Schnittstelle: 8 Systeme 2026, https://netzhandwerker.de/blog/zeiterfassung-datev-schnittstelle 14. Exportanleitung ADDISON - DATEV, https://www.datev.de/content/dam/markenassets/themen-und-produktgruppen/zielgruppen/steuerberater/rechnungswesen/exportanleitungen/addison-rechnungswesen-exportanleitung.pdf 15. DATEV-Schnittstelle für Ihre Finanzbuchhaltung - TOPIX, https://www.topix.de/de/software/schnittstellen/datev.html